

Woxsen University

School of Technology

PROJECT REPORT

on

PROPERTYDESK: PROPERTY RENTAL & LEASE MANAGEMENT SYSTEM

DBMS Course Project Report

Submitted in partial fulfillment of the requirements for the degree of

B.Tech. in Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

Submitted by

Mohammad Zeeshan – 25WU0102160

Academic Year: 2025-29

2. Abstract

PropertyDesk is a Property Rental & Lease Management System developed as a DBMS course project to replace scattered, manual rental records with a single, reliable relational database. Property managers who rely on registers and spreadsheets commonly face untracked rent, overlapping or invalid leases, scattered tenant and maintenance records, and little visibility of vacant units. PropertyDesk addresses these problems through a centralized MySQL 8.0 database, smartnest_db, comprising twelve inter-related tables covering owners, properties, units, tenants, leases, rent schedules, deposits, payments, inspections, maintenance requests, renewals and terminations.

The schema was designed from an Entity–Relationship model and normalized to Third Normal Form (3NF) to minimize redundancy and update anomalies. Data integrity is enforced through primary and foreign keys, unique, not-null and check constraints, together with triggers and a stored procedure that validate lease dates and prevent overlapping leases. Database views provide ready-made reports on vacant units, due rent, lease expiry, owner income and tenant history.

The front end is built with HTML, CSS and JavaScript, while a Python backend communicates with the database through MySQL Connector. The application supports complete CRUD (Create, Read, Update, Delete) operations for properties, units, tenants and leases, and provides modules for rent scheduling, payment recording, deposit handling, inspections, maintenance tracking, renewals and terminations. Operational reports are generated directly from SQL queries and views. The resulting system demonstrates how a well-designed relational database, supported by constraints and a simple application layer, can improve accuracy, traceability and reporting in rental and lease management.

3. Introduction and Problem Statement

3.1 Background

Property rental management involves maintaining information about property owners, properties, units, tenants, leases, rent schedules, payments, maintenance requests and other lease-related activities. When these records are maintained manually or across separate files, it becomes difficult to keep information consistent and updated.

PropertyDesk provides a centralized database-based system for managing these records. The system stores the information in related tables and provides a user interface for viewing and managing the data.

3.2 Problems in the Existing Approach

The main problems addressed by PropertyDesk are:

- Property and unit information may be maintained separately.
- Tenant and lease records can become difficult to track.
- Rent schedules and payments require regular monitoring.
- Maintenance requests need to be linked with the correct tenant and unit.

- Manual records can lead to duplicate or inconsistent information.
- Searching for specific records becomes time-consuming.
- Reports require information from multiple related records.

3.3 Proposed Solution

PropertyDesk combines the database and user interface into one system. The MySQL database stores the project data, while the application provides screens for properties, units, tenants, leases, payments, maintenance and reports.

The relational structure allows related information to be connected using primary keys and foreign keys. This makes it possible to retrieve combined information using SQL queries and database views.

4. Objectives and Scope

4.1 Detailed Objectives

The main objectives of PropertyDesk are:

1. To maintain property and unit information in a structured database.
2. To store tenant details and rental history.
3. To manage lease records and lease status.
4. To maintain rent schedules and payment information.
5. To record maintenance requests.
6. To provide useful database-based reports.
7. To provide CRUD operations through the application interface.
8. To maintain data integrity using database constraints.
9. To connect the application directly with the MySQL database.
10. To demonstrate practical implementation of DBMS concepts.

4.2 Functional Scope

The system covers:

- Property management
- Unit management
- Tenant management
- Lease management
- Rent and payment management
- Maintenance management
- Database reports
- Searching and filtering
- Record insertion

- Record viewing
- Record deletion
- Database validation

4.3 Users of the System

The system is mainly intended for property management activities. A property manager can use the application to maintain property records, tenant information, leases, payments, maintenance requests and reports.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Requirement
Operating System	Windows
Database	MySQL 8.0
Database Tool	MySQL Workbench
Backend	Python
Frontend	HTML5, CSS, JavaScript
Database Connectivity	MySQL Connector
Database Name	smartnest_db
Development Environment	Local development environment

5.2 Hardware Requirements

The system can be developed and executed on a standard computer with:

- Modern multi-core processor
- Minimum 8 GB RAM recommended
- Sufficient storage for source code and database
- Keyboard and mouse
- Display capable of running the browser and development tools

5.3 Technology Selection

MySQL 8.0 is used as the database because the project requires relational tables, constraints, foreign keys, views and SQL queries.

Python is used for backend processing and database communication. HTML, CSS and JavaScript are used to provide the application interface.

6. ER Diagram

The conceptual design of PropertyDesk models twelve entities connected by one-to-many relationships that follow the real-world flow of rental management: an owner owns properties, properties contain units, and units are let to tenants through leases. Each lease then generates rent schedules, payments, deposits and, where applicable, renewals or termination records.

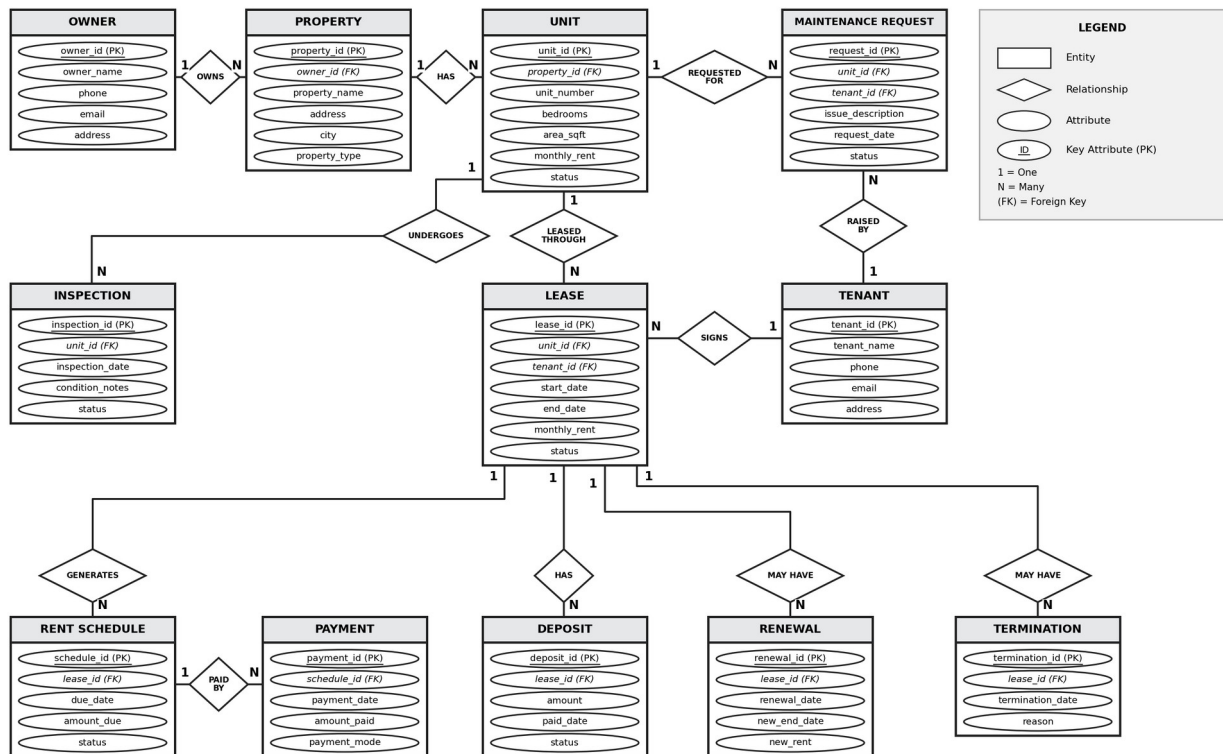


Figure 6.1: Entity-Relationship Diagram of PropertyDesk (smartnest_db)

Figure 6.1: Entity-Relationship Diagram of PropertyDesk (smartnest_db)

6.1 ER Diagram Interpretation

The Entity-Relationship Diagram represents the structure of the PropertyDesk database. It contains the main entities involved in property rental and lease management.

The major entities include OWNER, PROPERTY, UNIT, TENANT, LEASE, RENT_SCHEDULE, PAYMENT, DEPOSIT, INSPECTION, MAINTENANCE_REQUEST, RENEWAL and TERMINATION.

Each entity contains attributes that represent the information stored for that entity. Primary keys uniquely identify records, while foreign keys establish relationships between related entities.

The relationships show how records are connected. For example, an owner can have multiple properties, a property can contain multiple units, and a lease connects a tenant with a unit.

6.2 Relationship Interpretation

The main relationships are:

- OWNER → PROPERTY: one owner can own multiple properties.
- PROPERTY → UNIT: one property can contain multiple units.
- UNIT → LEASE: a unit can have lease records.
- TENANT → LEASE: a tenant can have lease records.
- LEASE → RENT_SCHEDULE: a lease generates rent schedules.
- RENT_SCHEDULE → PAYMENT: rent schedules can have payment records.
- LEASE → DEPOSIT: a lease can have deposit information.
- UNIT → INSPECTION: units can undergo inspections.
- UNIT → MAINTENANCE_REQUEST: maintenance requests are associated with units.
- TENANT → MAINTENANCE_REQUEST: maintenance requests are associated with tenants.
- LEASE → RENEWAL: leases can have renewal records.
- LEASE → TERMINATION: leases can have termination records.

6.3 Relationship Summary

The major relationships are summarized below. In each case the primary key of the parent entity appears as a foreign key in the child entity.

Relationship	Cardinality	Key Implementation
Owner → Property	1 : N	property.owner_id references owner.owner_id
Property → Unit	1 : N	unit.property_id references property.property_id
Unit → Lease	1 : N	lease.unit_id references unit.unit_id
Tenant → Lease	1 : N	lease.tenant_id references tenant.tenant_id
Lease → Rent Schedule	1 : N	rent_schedule.lease_id references lease.lease_id
Rent Schedule → Payment	1 : N	payment.schedule_id references rent_schedule.schedule_id

Relationship	Cardinality	Key Implementation
Lease → Deposit	1 : N	deposit.lease_id references lease.lease_id
Unit → Inspection	1 : N	inspection.unit_id references unit.unit_id
Unit / Tenant → Maintenance Request	1 : N each	maintenance_request.unit_id and tenant_id reference unit and tenant
Lease → Renewal	1 : N	renewal.lease_id references lease.lease_id
Lease → Termination	1 : N	termination.lease_id references lease.lease_id

Every entity has a single-attribute surrogate primary key, which uniquely identifies its records. Foreign keys enforce referential integrity, so a child record cannot exist without a valid parent. The Lease entity acts as the central link between Unit and Tenant, resolving the many-to-many association between them over time.

7. Relational Schema and Normalization

7.1 Relational Schema

The ER model is mapped to twelve relations. PK denotes the primary key and FK a foreign key.

Relation	Primary Key	Important Foreign Keys
OWNER	owner_id	—
PROPERTY	property_id	owner_id → OWNER
UNIT	unit_id	property_id → PROPERTY
TENANT	tenant_id	—
LEASE	lease_id	unit_id → UNIT; tenant_id → TENANT
RENT_SCHEDULE	schedule_id	lease_id → LEASE
DEPOSIT	deposit_id	lease_id → LEASE
PAYMENT	payment_id	schedule_id → RENT_SCHEDULE
INSPECTION	inspection_id	unit_id → UNIT
MAINTENANCE_REQUEST	request_id	unit_id → UNIT; tenant_id → TENANT
RENEWAL	renewal_id	lease_id → LEASE
TERMINATION	termination_id	lease_id → LEASE

7.2 Functional Dependencies

The principal functional dependencies, where each determinant is a primary key, are:

- owner_id → owner_name, phone, email, address
- property_id → owner_id, property_name, address, city, property_type
- unit_id → property_id, unit_number, bedrooms, area_sqft, monthly_rent, status
- tenant_id → tenant_name, phone, email, address
- lease_id → unit_id, tenant_id, start_date, end_date, monthly_rent, status

The remaining relations follow the same pattern: each key (schedule_id, deposit_id, payment_id, inspection_id, request_id, renewal_id, termination_id) determines all other attributes of its relation.

7.3 Normalization

Normal Form	Requirement	How PropertyDesk satisfies it
1NF	Atomic values; no repeating groups	Every column holds a single value. Repeating data such as multiple units, payments or inspections is stored as separate rows in their own tables.
2NF	1NF and no partial dependency on part of a key	All relations use single-column primary keys, so no non-key attribute can depend on part of a key.
3NF	2NF and no transitive dependency	Owner, property, unit and tenant details are stored once in their own relations and referenced by foreign keys, so non-key attributes depend only on the key.

For example, owner details are not repeated in PROPERTY and property details are not repeated in UNIT; they are reached through foreign keys. The final design is therefore normalized up to Third Normal Form (3NF), which reduces redundancy and avoids insertion, update and deletion anomalies.

7.4 First Normal Form

First Normal Form requires that table attributes contain atomic values and that repeating groups are avoided.

In PropertyDesk, information such as tenant details, property details and payment details is stored in separate columns. Each field represents a single value.

7.5 Second Normal Form

Second Normal Form requires that non-key attributes depend on the complete primary key.

The PropertyDesk tables separate information according to the entity it belongs to. For example, tenant information is maintained in the TENANT table rather than being repeatedly stored inside lease or payment records.

7.6 Third Normal Form

Third Normal Form removes unnecessary dependencies between non-key attributes.

PropertyDesk separates entities such as OWNER, PROPERTY, UNIT, TENANT and LEASE. This reduces repeated data and makes updates easier.

7.7 Benefits of Normalization

The normalized structure provides:

- Reduced data redundancy
- Better data consistency
- Easier updates

- Clear relationships between entities
- Better referential integrity
- Easier maintenance of the database

8. Data Dictionary

8.1 OWNER

Column	Data Type	Key / Constraint	Description
owner_id	INT	PK, AUTO_INCREMENT	Unique owner identifier
owner_name	VARCHAR(100)	NOT NULL	Name of the owner
phone	VARCHAR(15)	NOT NULL	Contact number
email	VARCHAR(100)	UNIQUE	Email address
address	VARCHAR(255)		Owner address

8.2 PROPERTY

Column	Data Type	Key / Constraint	Description
property_id	INT	PK, AUTO_INCREMENT	Unique property identifier
owner_id	INT	FK → OWNER, NOT NULL	Owner of the property
property_name	VARCHAR(100)	NOT NULL	Name of the property
address	VARCHAR(255)	NOT NULL	Property address
city	VARCHAR(50)	NOT NULL	City
property_type	VARCHAR(30)		Type of property

8.3 UNIT

Column	Data Type	Key / Constraint	Description
unit_id	INT	PK, AUTO_INCREMENT	Unique unit identifier
property_id	INT	FK → PROPERTY, NOT NULL	Property containing the unit
unit_number	VARCHAR(20)	NOT NULL	Unit number within property
bedrooms	INT	CHECK ≥ 0	Number of bedrooms
area_sqft	DECIMAL(8,2)	CHECK > 0	Unit area in sq. ft.
monthly_rent	DECIMAL(10,2)	NOT NULL, CHECK > 0	Listed monthly rent
status	VARCHAR(20)	DEFAULT Vacant	Occupancy status

8.4 TENANT

Column	Data Type	Key / Constraint	Description
tenant_id	INT	PK, AUTO_INCREMENT	Unique tenant identifier
tenant_name	VARCHAR(100)	NOT NULL	Name of the tenant
phone	VARCHAR(15)	NOT NULL	Contact number
email	VARCHAR(100)	UNIQUE	Email address
address	VARCHAR(255)		Permanent address

8.5 LEASE

Column	Data Type	Key / Constraint	Description
lease_id	INT	PK, AUTO_INCREMENT	Unique lease identifier
unit_id	INT	FK → UNIT, NOT NULL	Leased unit
tenant_id	INT	FK → TENANT, NOT NULL	Lessee
start_date	DATE	NOT NULL	Lease start date
end_date	DATE	NOT NULL, CHECK > start_date	Lease end date
monthly_rent	DECIMAL(10,2)	NOT NULL	Agreed monthly rent
status	VARCHAR(20)	DEFAULT Active	Lease status

8.6 RENT_SCHEDULE

Column	Data Type	Key / Constraint	Description
schedule_id	INT	PK, AUTO_INCREMENT	Unique schedule entry identifier
lease_id	INT	FK → LEASE, NOT NULL	Related lease
due_date	DATE	NOT NULL	Rent due date
amount_due	DECIMAL(10,2)	NOT NULL, CHECK > 0	Rent amount due
status	VARCHAR(20)	DEFAULT Pending	Payment status

8.7 DEPOSIT

Column	Data Type	Key / Constraint	Description
deposit_id	INT	PK, AUTO_INCREMENT	Unique deposit identifier
lease_id	INT	FK → LEASE, NOT NULL	Related lease
amount	DECIMAL(10,2)	NOT NULL, CHECK ≥ 0	Security deposit amount
paid_date	DATE		Date deposit received
status	VARCHAR(20)		Deposit status

8.8 PAYMENT

Column	Data Type	Key / Constraint	Description
payment_id	INT	PK, AUTO_INCREMENT	Unique payment identifier
schedule_id	INT	FK → RENT_SCHEDULE, NOT NULL	Rent instalment paid
payment_date	DATE	NOT NULL	Date of payment
amount_paid	DECIMAL(10,2)	NOT NULL, CHECK > 0	Amount paid
payment_mode	VARCHAR(30)		Mode of payment

8.9 INSPECTION

Column	Data Type	Key / Constraint	Description
inspection_id	INT	PK, AUTO_INCREMENT	Unique inspection identifier
unit_id	INT	FK → UNIT, NOT NULL	Inspected unit
inspection_date	DATE	NOT NULL	Date of inspection
condition_notes	VARCHAR(255)		Observed condition
status	VARCHAR(20)		Inspection status

8.10 MAINTENANCE_REQUEST

Column	Data Type	Key / Constraint	Description
request_id	INT	PK, AUTO_INCREMENT	Unique request identifier
unit_id	INT	FK → UNIT, NOT NULL	Unit needing maintenance
tenant_id	INT	FK → TENANT, NOT NULL	Tenant raising the request
issue_description	VARCHAR(255)	NOT NULL	Description of the issue
request_date	DATE	NOT NULL	Date raised
status	VARCHAR(20)	DEFAULT Open	Request status

8.11 RENEWAL

Column	Data Type	Key / Constraint	Description
renewal_id	INT	PK, AUTO_INCREMENT	Unique renewal identifier
lease_id	INT	FK → LEASE, NOT NULL	Lease renewed
renewal_date	DATE	NOT NULL	Date of renewal
new_end_date	DATE	NOT NULL	Extended end date
new_rent	DECIMAL(10,2)	CHECK > 0	Revised monthly rent

8.12 TERMINATION

Column	Data Type	Key / Constraint	Description
termination_id	INT	PK, AUTO_INCREMENT	Unique termination identifier
lease_id	INT	FK → LEASE, NOT NULL	Lease terminated
termination_date	DATE	NOT NULL	Date of termination
reason	VARCHAR(255)		Reason for termination

8.13 Constraint Interpretation

The database uses different constraints to maintain valid records.

Primary Key: Uniquely identifies each record in a table.

Foreign Key: Connects a record to a related record in another table.

NOT NULL: Ensures that required fields cannot remain empty.

UNIQUE: Prevents duplicate values where uniqueness is required.

DEFAULT: Provides a value automatically when a value is not supplied.

CHECK: Restricts values according to a defined condition.

AUTO_INCREMENT: Automatically generates sequential numeric identifiers for applicable primary keys.

These constraints help maintain the integrity of the PropertyDesk database.

9. SQL Commands Used

The database layer of PropertyDesk was implemented in MySQL 8.0 using standard SQL. This section summarizes the commands used; the complete script is maintained in the project repository.

9.1 DDL Commands

Command / Clause	Use in PropertyDesk
CREATE DATABASE	Creates the smartnest_db database.
CREATE TABLE	Defines the twelve relations.
PRIMARY KEY	Uniquely identifies each record in every table.
FOREIGN KEY	Links child tables to parents (e.g., lease to unit and tenant) and enforces referential integrity.
NOT NULL	Makes essential attributes mandatory.
UNIQUE	Prevents duplicate values in attributes such as email.
DEFAULT	Supplies initial values such as record status.
CHECK	Restricts values, for example positive rent and amounts, and end date later than start date.
CREATE INDEX	Speeds up frequent lookups on key and search columns.

Command / Clause	Use in PropertyDesk
CREATE VIEW	Defines the Vacant Units, Due Rent, Lease Expiry, Owner Income and Tenant History views.

9.2 DML Commands

Command	Use in PropertyDesk
INSERT	Adds sample data and new records entered through the application.
UPDATE	Modifies existing records, such as tenant details or the status of rent, leases and requests.
DELETE	Removes records that have no dependent data; deletion of referenced records is rejected by foreign keys.
SELECT	Retrieves data for screens, searches and reports (Section 10).

9.3 Database Logic

- Triggers: execute automatically on data changes to validate business rules, such as lease dates and overlapping leases, before the data is stored.
- Stored procedure: encapsulates a repeatable database operation inside MySQL so that the same logic is applied every time it is called.
- Referential integrity: foreign keys ensure every child record refers to an existing parent and block deletion of records still in use.
- Business-rule validation: CHECK constraints, triggers and the stored procedure together keep invalid or inconsistent data out of the database, independently of the application.

9.4 Sample Outputs

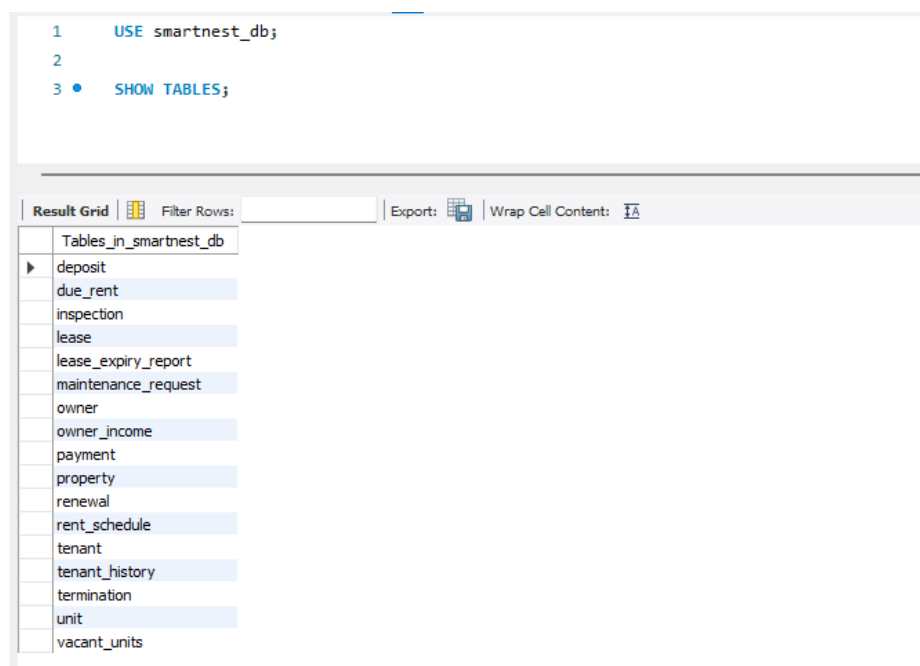


Figure 9.1: Tables available in the smartnest_db database.

The screenshot shows a database management interface. At the top, there's a toolbar with various icons and a dropdown menu set to 'Limit to 1000 rows'. Below the toolbar, the SQL editor contains two commands: `USE smartnest_db;` and `SELECT * FROM tenant;`. The main area displays a 'Result Grid' with a table of tenant records. The table has five columns: `tenant_id`, `name`, `phone`, `email`, and `id_proof_no`. It contains 17 rows of data, with the last row showing NULL values. The interface also includes options for 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'.

tenant_id	name	phone	email	id_proof_no
1	Aarav Mehta	9000011111	aarav@gmail.com	AADHAR 1001
2	Priya Singh	9000022222	priya.singh@gmail.com	AADHAR 1002
3	Rahul Verma	9000033333	rahul.verma@gmail.com	AADHAR 1003
4	Sneha Reddy	9000044444	sneha.reddy@gmail.com	AADHAR 1004
5	Karan Shah	9000055555	karan.shah@gmail.com	AADHAR 1005
6	Riya Kapoor	9000066666	riya.kapoor@gmail.com	AADHAR 1006
7	Aditya Rao	9000077777	aditya.rao@gmail.com	AADHAR 1007
8	Meera Nair	9000088888	meera.nair@gmail.com	AADHAR 1008
9	Varun Malhotra	9000099999	varun.malhotra@gmail.com	AADHAR 1009
10	Ishita Jain	9000012345	ishita.jain@gmail.com	AADHAR 1010
11	Rohan Das	9000023456	rohan.das@gmail.com	AADHAR 1011
12	Pooja Menon	9000034567	pooja.menon@gmail.com	AADHAR 1012
13	Nikhil Gupta	9000045678	nikhil.gupta@gmail.com	AADHAR 1013
14	Ananya Rao	9000056789	ananya.rao@gmail.com	AADHAR 1014
15	Siddharth Bose	9000067890	siddharth.bose@gmail.com	AADHAR 1015
16	Tanya Kapoor	9000078901	tanya.kapoor@gmail.com	AADHAR 1016
17	zeeshan	8885236266	mohdxeeshan1221@gmail...	23456789456
	NULL	NULL	NULL	NULL

Figure 9.2: Sample tenant records from the database.

9.5 DDL Commands

DDL commands are used to define the database structure. In PropertyDesk, DDL operations include creation of the database tables and definition of keys, constraints and indexes.

The main database objects are:

- OWNER
- PROPERTY
- UNIT
- TENANT
- LEASE
- RENT_SCHEDULE
- DEPOSIT
- PAYMENT
- INSPECTION
- MAINTENANCE_REQUEST
- RENEWAL
- TERMINATION

9.6 DML Commands

DML commands are used to work with the records stored in the database.

The project uses operations such as:

- INSERT — adding records
- SELECT — retrieving records

- UPDATE — modifying records
- DELETE — removing records

These operations are also demonstrated through the PropertyDesk user interface.

9.7 Database Integrity

Database constraints, foreign keys, triggers and stored procedures are used to enforce project rules.

The database also contains views that provide prepared results for reports.

9.8 SQL Implementation Summary

SQL is used throughout the project for database creation, record management, relationship handling, validation and report generation. The application interacts with these database operations through the backend.

10. Queries with Outputs

The project demonstrates the following query categories against smartnest_db.

Category	Purpose in PropertyDesk
JOIN queries	Combine related tables, e.g., a lease with its tenant and unit.
Multi-table JOIN	Combine owner, property, unit, lease and tenant data in one result.
Aggregate functions	Summarize amounts and counts, such as total rent collected.
GROUP BY	Group results by owner, property or status.
Subqueries	Select records based on the result of another query.
Filtering	Restrict results with WHERE conditions such as status or date.
Views	Stored queries that provide reusable reports.
Reporting queries	Operational reports for management, listed below.

Report	Source	Information Provided
Vacant Units	Vacant Units view	Units currently available for lease
Due Rent	Due Rent view	Rent instalments that are due or unpaid
Lease Expiry	Lease Expiry view	Leases approaching or past their end date
Owner Income	Owner Income view	Income received per owner
Tenant History	Tenant History view	Past and present leases of each tenant
Maintenance	Maintenance request query	Maintenance requests with their unit, tenant and status

10.1 Query Given During Presentation-II

Query Objective

To retrieve tenant details together with their corresponding lease information by using INNER JOIN between the related database tables.

SQL Query

```
USE smartnest_db;

SELECT
    t.tenant_id,
    t.name AS tenant_name,
    l.lease_id,
    l.unit_id,
    l.start_date,
    l.end_date,
    l.monthly_rent,
    l.status
FROM tenant t
INNER JOIN lease l
    ON t.tenant_id = l.tenant_id
INNER JOIN unit u
    ON l.unit_id = u.unit_id;
```

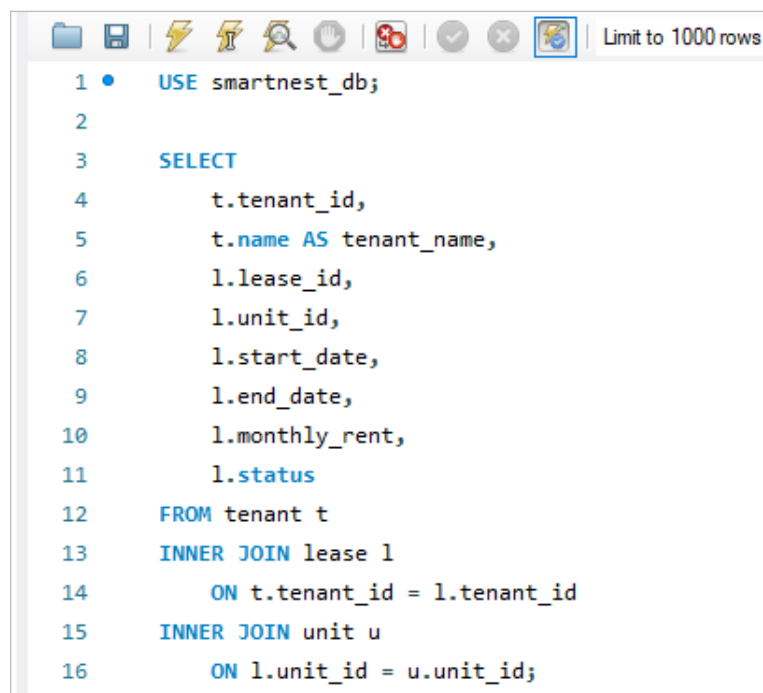


Figure 10.1: Presentation-II JOIN query

Query Explanation

The query uses INNER JOIN to combine related records from the TENANT, LEASE and UNIT tables. The relationship between TENANT and LEASE is established through tenant_id, while the LEASE and UNIT tables are related through unit_id.

The result displays the tenant ID, tenant name, lease ID, unit ID, lease dates, monthly rent and lease status.

Result: 18 matching records were returned.

Result Grid								
Filter Rows:			Export:		Wrap Cell Content:			
	tenant_id	tenant_name	lease_id	unit_id	start_date	end_date	monthly_rent	status
▶	1	Aarav Mehta	1	1	2026-01-01	2026-12-31	25000.00	Active
	2	Priya Singh	2	2	2026-02-01	2027-01-31	22000.00	Active
	3	Rahul Verma	3	3	2026-03-01	2027-02-28	32000.00	Active
	3	Rahul Verma	17	18	2025-01-01	2025-12-31	18000.00	Terminated
	4	Sneha Reddy	4	4	2026-04-15	2026-10-15	28000.00	Active
	4	Sneha Reddy	18	17	2024-06-01	2025-05-31	16000.00	Terminated
	5	Karan Shah	5	5	2026-05-01	2027-04-30	24000.00	Active
	6	Riya Kapoor	6	6	2026-06-01	2027-05-31	26000.00	Active
	7	Aditya Rao	7	7	2026-01-15	2026-10-05	18000.00	Active
	8	Meera Nair	8	8	2026-07-01	2027-06-30	30000.00	Active
	9	Varun Malhotra	9	9	2026-08-01	2027-07-31	20000.00	Active
	10	Ishita Jain	10	10	2026-01-01	2026-12-31	27000.00	Active
	11	Rohan Das	11	11	2026-09-01	2027-08-31	21000.00	Active
	12	Pooja Menon	12	12	2026-09-15	2027-09-14	23000.00	Active
	13	Nikhil Gupta	13	13	2026-01-01	2026-10-20	19000.00	Active
	14	Ananya Rao	14	14	2026-01-01	2027-03-31	31000.00	Active
	15	Siddharth Bose	15	15	2026-01-01	2026-10-10	17000.00	Active
	16	Tanya Kapoor	16	19	2026-04-01	2027-03-31	20000.00	Active

Figure 10.2: Output of the Presentation-II JOIN query

11. UI Design and Screenshots

PropertyDesk is the working application developed for Review 3. Its HTML, CSS and JavaScript interface is connected to the smartnest_db database through the Python backend, so every screen reads and writes real data in MySQL. The interface is organized into modules corresponding to the main entities and reports of the system.

11.1 Dashboard

The Dashboard provides an overview of the property portfolio. It displays information such as total properties, total units, available units, occupied units, tenants and active leases.

The dashboard also provides quick actions and recent lease/payment activity.

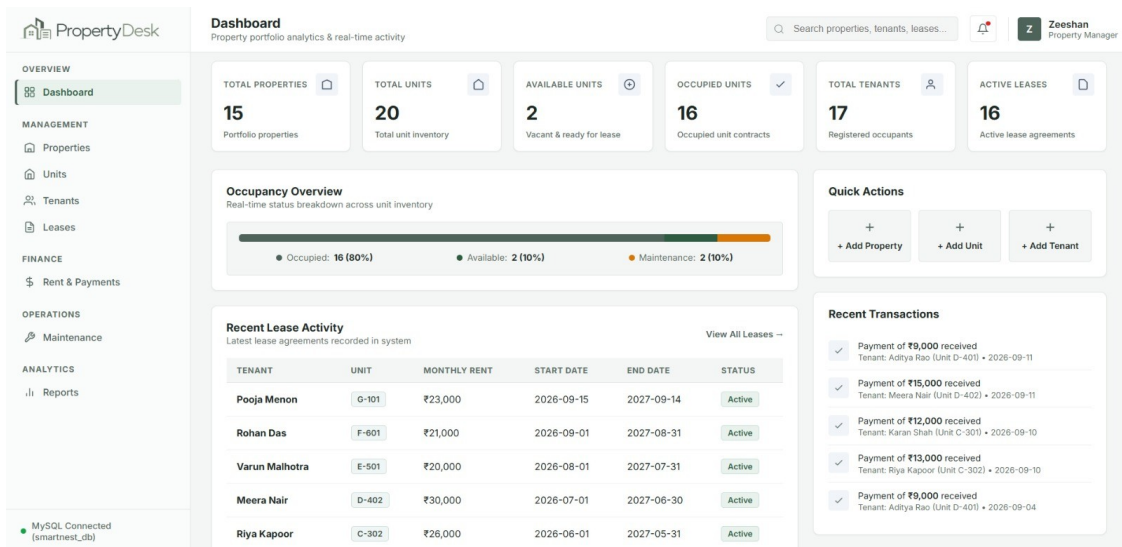


Figure 11.1: PropertyDesk Dashboard

11.2 Properties

The Properties module displays property records with owner, address, city, property type and unit information.

Users can search and filter property records and access available actions from the interface.

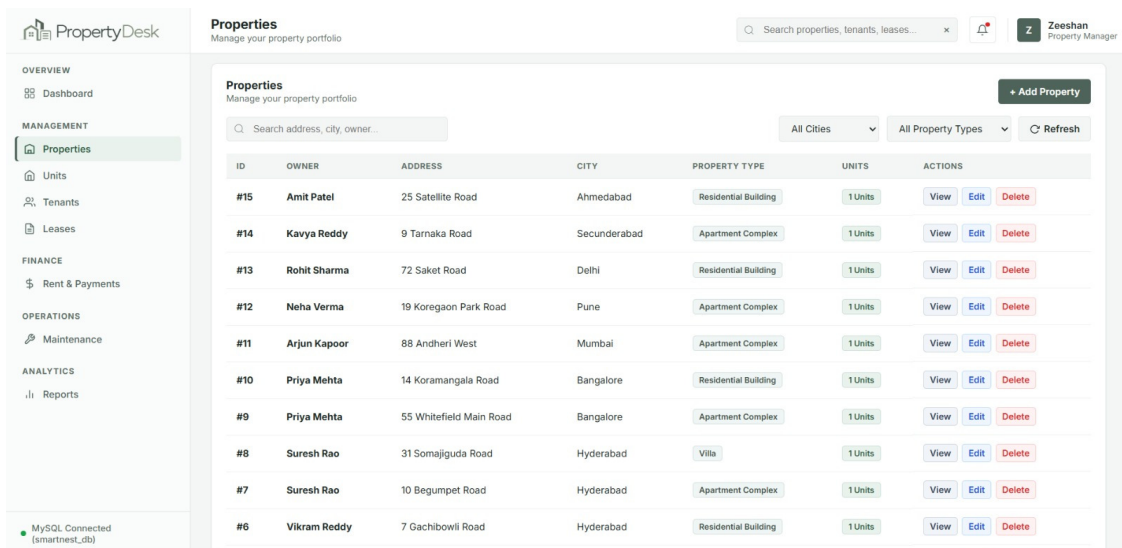


Figure 11.2: Properties Management

11.3 Units

The Units module displays units associated with properties. It includes information such as unit number, property, floor, bedrooms, monthly rent and status.

The status helps distinguish available, occupied and maintenance units.

UNIT ID	PROPERTY ADDRESS	UNIT NO	FLOOR	BEDROOMS	MONTHLY RENT	STATUS	ACTIONS
#20	25 Satellite Road (Ahmedabad)	O-301	Floor 3	2 BHK	₹20,000	Maintenance	View Edit Delete
#19	9 Tarnaka Road (Secunderabad)	N-201	Floor 2	2 BHK	₹20,000	Occupied	View Edit Delete
#18	72 Saket Road (Delhi)	M-101	Floor 1	2 BHK	₹24,000	Maintenance	View Edit Delete
#17	19 Koregaon Park Road (Pune)	L-601	Floor 6	2 BHK	₹26,000	Maintenance	View Edit Delete
#16	88 Andheri West (Mumbai)	K-501	Floor 5	3 BHK	₹29,000	Available	View Edit Delete
#15	14 Koramangala Road (Bangalore)	J-401	Floor 4	2 BHK	₹17,000	Occupied	View Edit Delete
#14	55 Whitefield Main Road (Bangalore)	I-301	Floor 3	3 BHK	₹31,000	Occupied	View Edit Delete
#13	31 Somajiguda Road (Hyderabad)	H-201	Floor 2	1 BHK	₹19,000	Occupied	View Edit Delete
#12	10 Begumpet Road (Hyderabad)	G-101	Floor 1	2 BHK	₹23,000	Occupied	View Edit Delete
#11	7 Gachibowli Road (Hyderabad)	F-601	Floor 6	2 BHK	₹21,000	Occupied	View Edit Delete

Figure 11.3: Units Management

11.4 Tenants

The Tenants module stores and displays tenant information such as tenant ID, name, phone, email and ID proof number.

The interface supports viewing and deleting tenant records and adding new tenant records.

TENANT ID	NAME	PHONE	EMAIL	ID PROOF NO.	ACTIONS
#16	Tania Kapoor	9000078922	tania.kapoor@gmail.com	AADHAR1016	View Edit Delete
#15	Siddharth Bose	9000067890	siddharth.bose@gmail.com	AADHAR1015	View Edit Delete
#14	Ananya Rao	9000056789	ananya.rao@gmail.com	AADHAR1014	View Edit Delete
#13	Nikhil Gupta	9000045678	nikhil.gupta@gmail.com	AADHAR1013	View Edit Delete
#12	Pooja Menon	9000034567	pooja.menon@gmail.com	AADHAR1012	View Edit Delete
#11	Rohan Das	9000023456	rohan.das@gmail.com	AADHAR1011	View Edit Delete
#10	Ishita Jain	9000012345	ishita.jain@gmail.com	AADHAR1010	View Edit Delete
#9	Varun Malhotra	9000099999	varun.malhotra@gmail.com	AADHAR1009	View Edit Delete
#8	Meera Nair	9000088888	meera.nair@gmail.com	AADHAR1008	View Edit Delete
#7	Aditya Rao	9000077777	aditya.rao@gmail.com	AADHAR1007	View Edit Delete

Figure 11.4: Tenant Management

11.5 Leases

The Leases module displays lease information including tenant, unit, property, start date, end date, monthly rent, deposit and status.

This provides a central location for monitoring lease records.

PropertyDesk

OVERVIEW

Dashboard

MANAGEMENT

Properties

Units

Tenants

Leases

FINANCE

Rent & Payments

OPERATIONS

Maintenance

ANALYTICS

Reports

MySQL Connected (smartnest_db)

Leases

Manage rental agreements and lease lifecycle

Search properties, tenants, leases...

Zeeshan Property Manager

+ Create Lease

Refresh

Search by tenant, unit, address...

LEASE ID	TENANT	UNIT NO	PROPERTY	START DATE	END DATE	MONTHLY RENT	DEPOSIT	STATUS	ACTIONS
#18	Sneha Reddy 9000044444	L-601	19 Koregaon Park Road	2024-06-01	2025-05-31	₹16,000	₹32,000	Terminated	View
#17	Rahul Verma 9000033333	M-101	72 Saket Road	2025-01-01	2025-12-31	₹18,000	₹36,000	Terminated	View
#16	Tanya Kapoor 9000078901	N-201	9 Tarnaka Road	2026-04-01	2027-03-31	₹20,000	₹40,000	Active	View
#15	Siddharth Bose 9000067890	J-401	14 Koramangala Road	2026-01-01	2026-10-10	₹17,000	₹34,000	Active	View
#14	Ananya Rao 9000056789	I-301	55 Whitefield Main Road	2026-01-01	2027-03-31	₹31,000	₹62,000	Active	View
#13	Nikhil Gupta 9000045678	H-201	31 Somajiguda Road	2026-01-01	2026-10-20	₹19,000	₹38,000	Active	View
#12	Pooja Menon 9000034567	G-101	10 Begumpet Road	2026-09-15	2027-09-14	₹23,000	₹46,000	Active	View
#11	Rohan Das 9000023456	F-601	7 Gachibowli Road	2026-09-01	2027-08-31	₹21,000	₹42,000	Active	View
#10	Ishita Jain 9000012345	E-502	22 Hitech City Road	2026-01-01	2026-12-31	₹27,000	₹54,000	Active	View

Figure 11.5: Lease Management

11.6 Rent & Payments

The Rent & Payments module displays rent schedules and payment information. It includes due dates, amount due, payment status, amount paid, payment date and payment mode.

PropertyDesk

OVERVIEW

Dashboard

MANAGEMENT

Properties

Units

Tenants

Leases

FINANCE

Rent & Payments

OPERATIONS

Maintenance

ANALYTICS

Reports

MySQL Connected (smartnest_db)

Rent & Payments

Rent schedules and payment transactions

Search properties, tenants, leases.

Zeeshan Property Manager

Search by tenant or unit...

All Statuses

Refresh

SCHEDULE ID	TENANT	UNIT	DUE DATE	AMOUNT DUE	STATUS	AMOUNT PAID	PAYMENT DATE	MODE
#2	Aarav Mehta	A-101	2026-10-01	₹25,000	Pending	-	-	-
#4	Priya Singh	A-102	2026-10-01	₹22,000	Pending	-	-	-
#6	Rahul Verma	B-201	2026-10-01	₹32,000	Pending	-	-	-
#8	Sneha Reddy	B-202	2026-10-01	₹28,000	Pending	-	-	-
#10	Karan Shah	C-301	2026-10-01	₹24,000	Pending	₹12,000	2026-09-10	UPI
#12	Riya Kapoor	C-302	2026-10-01	₹26,000	Pending	₹13,000	2026-09-10	Bank Transfer
#14	Aditya Rao	D-401	2026-10-01	₹18,000	Pending	₹9,000	2026-09-11	UPI
#16	Meera Nair	D-402	2026-10-01	₹30,000	Pending	₹15,000	2026-09-11	Card
#18	Varun Malhotra	E-501	2026-10-01	₹20,000	Pending	-	-	-
#20	Ishita Jain	E-502	2026-10-01	₹27,000	Pending	-	-	-
#22	Rohan Das	F-601	2026-10-01	₹21,000	Pending	-	-	-

Figure 11.6: Rent & Payments

11.7 Maintenance

The Maintenance module records service requests and property maintenance activities. The interface displays the tenant, unit, property, request date, description, status and cost.

ID	TENANT	UNIT	PROPERTY	REQUEST DATE	DESCRIPTION	STATUS	COST	ACTIONS
#10	Tanya Kapoor	N-201	9 Tarnaka Road	2026-08-25	Balcony light replacement	Open	₹600	View
#9	Nikhil Gupta	H-201	31 Somajiguda Road	2026-08-23	Kitchen sink leakage	Completed	₹1,100	View
#8	Ishita Jain	E-502	22 Hitech City Road	2026-08-21	AC cooling problem	In Progress	₹3,000	View
#7	Varun Malhotra	E-501	22 Hitech City Road	2026-08-19	Electrical socket issue	Open	₹700	View
#6	Aditya Rao	D-401	21 Kondapur Road	2026-08-16	Door lock replacement	Completed	₹1,800	View
#5	Karan Shah	C-301	45 Madhapur Main Road	2026-08-13	Water purifier issue	Completed	₹900	View
#4	Sneha Reddy	B-202	18 Banjara Hills Road	2026-08-11	Bedroom fan making noise	Open	₹1,200	View
#3	Rahul Verma	B-201	18 Banjara Hills Road	2026-08-08	Kitchen exhaust not working	In Progress	₹1,500	View
#2	Priya Singh	A-102	12 Jubilee Hills Road	2026-08-04	Bathroom tap leaking	Completed	₹800	View
#1	Aarav Mehta	A-101	12 Jubilee Hills Road	2026-08-02	AC servicing required	Completed	₹2,500	View

Figure 11.7: Maintenance Management

11.8 Reports

The Reports module provides database-based reports including vacant units and due rent information.

The reports are generated from database views and provide summarized information to the user.

UNIT ID	PROPERTY	CITY	UNIT NO	FLOOR	BEDROOMS	RENT
#16	88 Andheri West	Mumbai	K-501	5	3 BHK	₹29,000
#20	25 Satellite Road	Ahmedabad	O-301	3	1 BHK	₹16,000

SCHEDULE ID	TENANT	DUE DATE	AMOUNT DUE	STATUS
#2	Aarav Mehta	2026-10-01	₹25,000	Pending
#4	Priya Singh	2026-10-01	₹22,000	Pending
#6	Rahul Verma	2026-10-01	₹32,000	Pending
#8	Sneha Reddy	2026-10-01	₹28,000	Pending
#9	Karan Shah	2026-09-01	₹24,000	Partial
#10	Karan Shah	2026-10-01	₹24,000	Pending
#11	Riya Kapoor	2026-09-01	₹26,000	Partial

Figure 11.8: Reports & Analytics

11.9 Search and Filtering

Search and filtering options are provided in several modules. They allow users to find relevant records without manually checking the complete table.

11.10 CRUD Operations

The application demonstrates database CRUD operations through the user interface.

- Create — adding records

- Read — viewing records
- Update — modifying applicable records
- Delete — removing records

The CRUD operations performed through the interface are verified against the MySQL database and presented as testing evidence in Section 13.

11.11 UI-to-Database Interaction

The application is connected to the smartnest_db MySQL database. User actions are processed by the backend and corresponding database operations are performed.

This allows changes made through the interface to be reflected in the database.

12. Implementation Details

12.1 System Architecture

PropertyDesk follows a simple application-to-database architecture:

User → Frontend → Python Backend → MySQL Database → Backend Response → Frontend

The frontend handles user interaction, the backend processes requests and the MySQL database stores the project data.

The technologies used in the application are summarized below.

Layer	Technology
Database	MySQL 8.0 (smartnest_db)
Backend	Python
Frontend	HTML, CSS, JavaScript
Connectivity	MySQL Connector

12.2 Frontend Implementation

The frontend is developed using HTML5, CSS and JavaScript. It provides pages for the major modules of PropertyDesk and displays records retrieved from the backend.

12.3 Backend Implementation

The Python backend handles application requests and communicates with the MySQL database. It processes CRUD operations and returns the required data to the frontend.

12.4 Database Connectivity

The backend uses MySQL connectivity to communicate with the smartnest_db database.

The connection allows the application to execute SQL operations and retrieve database results.

```
DB_CONFIG = {
    'host': '127.0.0.1',
    'port': 3306,
    'user': 'root',
    'password': '*****',
    'database': 'smartnest_db',
    'autocommit': True
}

def get_db_connection():
    """Create and return a MySQL connection."""
    return mysql.connector.connect(**DB_CONFIG)
```

Figure 12.1: Database connection code

12.5 CRUD Implementation

CRUD functionality connects the user interface with database operations.

For example, when a new tenant is added through the interface, the backend inserts the record into the TENANT table. When the record is deleted, the corresponding database record is removed if database constraints allow the operation.

12.6 Validation and Error Handling

Validation is applied to prevent invalid database operations. Database constraints and backend handling work together to prevent invalid values and maintain referential integrity.

12.7 Database Views and Reports

The application uses database views to provide prepared information for report screens. The project includes views such as:

- vacant_units
- due_rent
- lease_expiry_report
- owner_income
- tenant_history

12.8 Key Implementation Features

Feature	Description
CRUD operations	Create, view, update and delete records for the main entities.
Search / filtering	Retrieve records matching user-selected criteria.
Database validation	Constraints reject invalid values at the database level.
Foreign-key protection	Records still referenced by other tables cannot be deleted.
Triggers / stored procedure	Enforce lease rules and reusable logic inside MySQL.
Database views	Supply the report screens with ready-made queries.
Error handling	Database errors are caught by the backend and reported to the user.

13. Testing

13.1 Testing Strategy

Testing was performed to verify that the application, database and their interaction work correctly.

The testing focused on:

- Database connectivity
- CRUD operations
- Validation
- Foreign-key relationships
- UI operations
- Reports
- Database synchronization

13.2 Functional Testing

Functional testing checks whether the main application functions perform their intended operations.

Examples include adding tenants, viewing records, deleting records, loading reports and retrieving database information.

13.3 Database Testing

Database testing verifies that records inserted through the application are stored correctly and that deleted records are removed from the database when permitted.

13.4 CRUD Testing

CRUD testing was demonstrated using the tenant module.

A temporary tenant record was inserted through the UI, viewed in the application, deleted through the UI and then verified in MySQL.

13.5 Validation Testing

Validation testing checks that invalid operations are rejected. Examples include invalid values, invalid lease dates and operations that violate database rules.

13.6 Referential Integrity Testing

Foreign-key protection was tested to ensure that records referenced by other tables cannot be removed incorrectly.

13.7 UI Testing

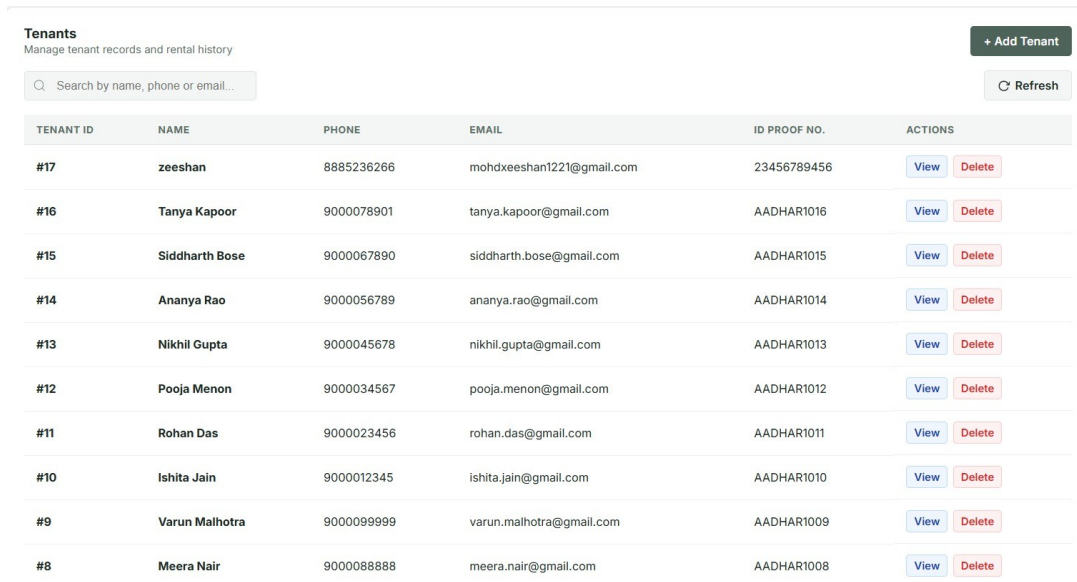
The major application screens were checked for correct display of database information and basic interaction.

13.8 Test Results Summary

The following test cases verify the main validations and operations of PropertyDesk. Actual results and status are to be recorded from the tests performed on the final application.

No.	Test Case	Expected Result	Actual Result	Status
1	Valid record insertion	Record is saved and displayed	Record inserted and displayed successfully	Pass
2	Tenant deletion with no dependencies	Tenant is deleted	Test Tenant deleted successfully	Pass
3	Foreign-key protected deletion	Deletion rejected with an error	Deletion rejected because the referenced record exists	Pass
4	Invalid lease dates	Lease is rejected	Invalid lease was rejected by validation	Pass
5	Negative rent / deposit	Entry is rejected	Invalid value was rejected	Pass
6	Leasing an unavailable unit	Lease is rejected	Lease creation was rejected	Pass
7	Payment exceeding amount due	Payment is rejected	Payment was rejected by validation	Pass
8	Report / view loading	Report displays data from the view	Report displayed data from the database view	Pass
9	Database connectivity	Application connects to smartnest_db	Application connected successfully to smartnest_db	Pass
10	CRUD synchronization with MySQL	Changes appear in the database	UI changes were reflected in MySQL	Pass

13.9 CRUD Demonstration Evidence



TENANT ID	NAME	PHONE	EMAIL	ID PROOF NO.	ACTIONS
#17	zeeshan	8885236266	mohdxeeshan1221@gmail.com	23456789456	View Delete
#16	Tanya Kapoor	9000078901	tanya.kapoor@gmail.com	AADHAR1016	View Delete
#15	Siddharth Bose	9000067890	siddharth.bose@gmail.com	AADHAR1015	View Delete
#14	Ananya Rao	9000056789	ananya.rao@gmail.com	AADHAR1014	View Delete
#13	Nikhil Gupta	9000045678	nikhil.gupta@gmail.com	AADHAR1013	View Delete
#12	Pooja Menon	9000034567	pooja.menon@gmail.com	AADHAR1012	View Delete
#11	Rohan Das	9000023456	rohan.das@gmail.com	AADHAR1011	View Delete
#10	Ishita Jain	9000012345	ishita.jain@gmail.com	AADHAR1010	View Delete
#9	Varun Malhotra	9000099999	varun.malhotra@gmail.com	AADHAR1009	View Delete
#8	Meera Nair	9000088888	meera.nair@gmail.com	AADHAR1008	View Delete

Figure 13.1: Tenant records before insertion.

Tenants

Manage tenant records and rental history

+ Add Tenant

Q Search by name, phone or email...

Refresh

TENANT ID	NAME	PHONE	EMAIL	ID PROOF NO.	ACTIONS
#20	Test Tenant	9876543210	testtenant@gmail.com	AADHAR1099	View Delete
#17	zeeshan	8885236266	mohdxeeshan1221@gmail.com	23456789456	View Delete
#16	Tanya Kapoor	9000078901	tanya.kapoor@gmail.com	AADHAR1016	View Delete
#15	Siddharth Bose	9000067890	siddharth.bose@gmail.com	AADHAR1015	View Delete
#14	Ananya Rao	9000056789	ananya.rao@gmail.com	AADHAR1014	View Delete
#13	Nikhil Gupta	9000045678	nikhil.gupta@gmail.com	AADHAR1013	View Delete
#12	Pooja Menon	9000034567	pooja.menon@gmail.com	AADHAR1012	View Delete
#11	Rohan Das	9000023456	rohan.das@gmail.com	AADHAR1011	View Delete
#10	Ishita Jain	9000012345	ishita.jain@gmail.com	AADHAR1010	View Delete
#9	Varun Malhotra	9000099999	varun.malhotra@gmail.com	AADHAR1009	View Delete

Figure 13.2: Test Tenant successfully inserted through the UI.

Query 1 SQL File 2* SQL File 3*

Limit to 1000 rows

```

1 • USE smartnest_db;
2 • SELECT * FROM tenant;

```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: T

	tenant_id	name	phone	email	id_proof_no
1	1	Aarav Mehta	9000011111	aarav@gmail.com	AADHAR.1001
2	2	Priya Singh	9000022222	priya.singh@gmail.com	AADHAR.1002
3	3	Rahul Verma	9000033333	rahul.verma@gmail.com	AADHAR.1003
4	4	Sneha Reddy	9000044444	sneha.reddy@gmail.com	AADHAR.1004
5	5	Karan Shah	9000055555	karan.shah@gmail.com	AADHAR.1005
6	6	Riya Kapoor	9000066666	riya.kapoor@gmail.com	AADHAR.1006
7	7	Aditya Rao	9000077777	aditya.rao@gmail.com	AADHAR.1007
8	8	Meera Nair	9000088888	meera.nair@gmail.com	AADHAR.1008
9	9	Varun Malhotra	9000099999	varun.malhotra@gmail.com	AADHAR.1009
10	10	Ishita Jain	9000012345	ishita.jain@gmail.com	AADHAR.1010
11	11	Rohan Das	9000023456	rohan.das@gmail.com	AADHAR.1011
12	12	Pooja Menon	9000034567	pooja.menon@gmail.com	AADHAR.1012
13	13	Nikhil Gupta	9000045678	nikhil.gupta@gmail.com	AADHAR.1013
14	14	Ananya Rao	9000056789	ananya.rao@gmail.com	AADHAR.1014
15	15	Siddharth Bose	9000067890	siddharth.bose@gmail.com	AADHAR.1015
16	16	Tanya Kapoor	9000078901	tanya.kapoor@gmail.com	AADHAR.1016
17	17	zeeshan	8885236266	mohdxeeshan1221@gmail...	23456789456
*	NULL	NULL	NULL	NULL	NULL

Figure 13.3: MySQL tenant table after the Test Tenant deletion.

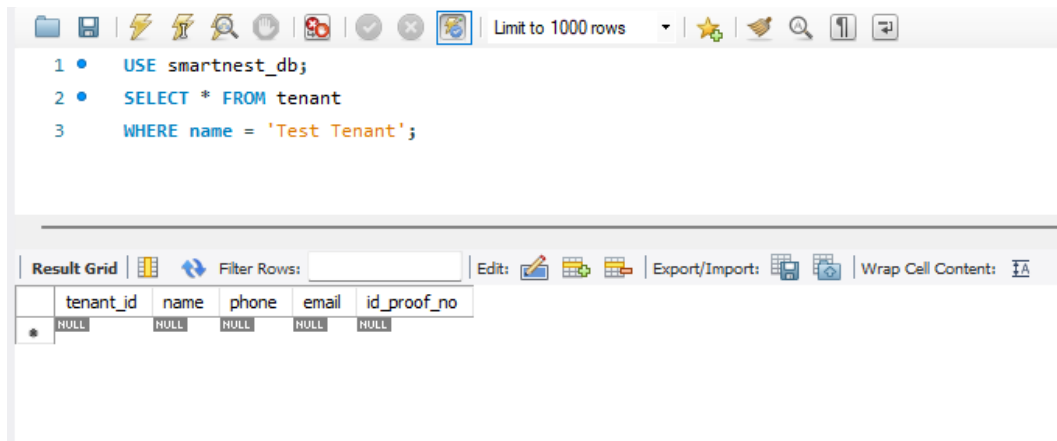


Figure 13.4: MySQL verification that Test Tenant no longer exists.

14. Conclusion and Future Enhancements

14.1 Project Achievements

PropertyDesk demonstrates the implementation of a relational database system for property rental and lease management.

The project integrates database design, normalization, SQL, constraints, relationships, database views and a working user interface.

The application provides separate modules for property, unit, tenant, lease, payment, maintenance and reporting activities.

14.2 Current Limitations

The current implementation focuses on the core property rental and lease management functions required for the project. Additional production-level features such as advanced authentication, online payment integration and automated notifications are outside the current implementation.

14.3 Future Enhancements

Possible future enhancements include:

- Role-based authentication
- Automated rent reminders
- Email/SMS notifications
- Online payment integration
- Digital document storage
- Advanced analytics
- Cloud deployment
- Mobile application support

These features can be considered for future versions of PropertyDesk.

15. References

- [1] Oracle. MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [2] Oracle. MySQL 8.0 Reference Manual — JOIN Clause. <https://dev.mysql.com/doc/refman/8.0/en/join.html>
- [3] Python Documentation. Python Software Foundation. Official Python documentation.
- [4] Oracle. MySQL Connector/Python Developer Guide. <https://dev.mysql.com/doc/connector-python/en/>
- [5] DBMS course material and project instructions, Woxsen University, 2025–2026.

16. Appendix

16.1 GitHub Repository

PropertyDesk DBMS Project Repository:

<https://github.com/Zeeshan0079/DBMS>

The repository contains the project source code, database files and presentation documents.

16.2 Additional Materials

The repository contains:

- Presentation-I
- Presentation-II
- Presentation-III
- Project Report
- Source Code
- Database / SQL files
- UI screenshots